

AI Web Tester — Development Work Log

Date: 25 May 2026 | Project: C:\Users\biswa\OneDrive\Desktop\AI assistant tool\ai-web-tester | Stack: FastAPI + Playwright + Next.js

1. Playwright Locator Fixes (browser.py)

Problem

Tests were failing with `Timeout 8000ms exceeded` on multiple elements. The errors included:

- `get_by_role("link", name="Explore Now").first` — timed out
- `get_by_role("link", name="Enquire Now").first` — timed out
- `Page.click: Timeout 10000ms exceeded` — waiting for `input[name='fullName']`
- `get_by_role("link", name="button:contains('Enquire Now)').first` — CSS passed as text

Root Causes Identified

- The `click()` method only tried 5 strategies; all failed for non-standard elements
- CSS selectors like `button:contains('X')` were being passed to `get_by_role(name=...)` — treating CSS as literal text
- `:contains()` is jQuery syntax — Playwright uses `:has-text()`
- Elements off-screen were never scrolled into view before clicking
- `type()` method had no fallback strategies — a single failed `fill()` caused the test to abort

Fixes Applied

Method	Change	Why
<code>click()</code>	Added <code>_is_css_selector()</code> detector + <code>_normalize_selector()</code>	Routes CSS selectors to locator-based strategies; plain text to role/text strategies
<code>click()</code>	Added <code>exact=False</code> role variants	Handles partial name matches
<code>click()</code>	Added <code>_scroll_and_force_click_css/text()</code>	Scrolls element into view, then force-clicks to bypass overlapping elements
<code>_normalize_selector()</code>	Converts <code>:contains('X')</code> → <code>:has-text('X')</code>	jQuery syntax not supported in Playwright
<code>type()</code>	CSS vs text routing, multiple fill strategies	Prevents CSS selector being searched as a placeholder/label name
<code>type()</code>	Added <code>_scroll_and_fill_css/text()</code>	Force-fills off-screen or covered inputs
Per-strategy timeout	Reduced 8000ms → 5000ms	Faster cascade through strategies when elements don't match

```
# CSS detection logic added to BrowserController
def _is_css_selector(self, selector: str) -> bool:
    return bool(re.search(r'[\#(>+~)]|^\.|:[\w-]+\(', selector))

def _normalize_selector(self, selector: str) -> str:
    return re.sub(r":contains\(['\"](.+?)['\"]\)", r":has-text('␣')", selector)
```

2. AI Model — Groq → Anthropic Claude

History

Stage	Model	Reason
Original	<code>llama-3.3-70b-versatile</code> (Groq)	Initial implementation
Switch attempt	<code>llama-3.1-8b-instant</code> (Groq)	Hit 100K tokens/day limit on 70b
Reverted	<code>llama-3.3-70b-versatile</code>	8b-instant caused browser hang + slow tests
Final	<code>claude-haiku-4-5-20251001</code> (Anthropic)	More reliable, better JSON output, no daily TPD limit

agent.py Changes

Before (Groq)	After (Anthropic)
<code>from groq import Groq</code>	<code>from anthropic import Anthropic</code>
<code>os.getenv("GROQ_API_KEY")</code>	<code>os.getenv("ANTHROPIC_API_KEY")</code>
<code>Groq(api_key=...)</code>	<code>Anthropic(api_key=...)</code>
<code>client.chat.completions.create(messages=[{system}, ...])</code>	<code>client.messages.create(system=SYSTEM_PROMPT, messages=[...])</code>
<code>response.choices[0].message.content</code>	<code>response.content[0].text</code>
Sync call blocks event loop	<code>await asyncio.to_thread(...)</code> — non-blocking

Key difference: Anthropic's API separates the system prompt from the messages list. The conversation also must always start with a user turn — trimming logic was updated accordingly.

```
# Final API call in agent.py
trimmed = conversation[-4:]
if trimmed and trimmed[0]["role"] != "user":
    trimmed = trimmed[1:]

response = await asyncio.to_thread(
    self.client.messages.create,
    model=self.model,          # claude-haiku-4-5-20251001
    max_tokens=512,
    system=SYSTEM_PROMPT,     # separate from messages
    messages=trimmed,
)
ai_text = response.content[0].text.strip()
```

3. Headed Mode Fix (browser.py)

Problem

When running tests in headed mode (visible browser window), the browser never started. The frontend showed "Waiting for browser to start..." indefinitely.

Root Causes

- Playwright's bundled Chromium binary can be flagged by Windows Defender when launched visibly (headed), causing it to hang
- `--disable-gpu` flag was preventing the browser window from rendering
- No launch timeout — a hung browser gave no error feedback

Fix

Switched headed mode to use `channel="msedge"` — Microsoft Edge is built into Windows 11, always trusted by Defender, and requires no extra installation.

```
async def start(self):
    self.playwright = await async_playwright().start()
    if self.headless:
        self.browser = await self.playwright.chromium.launch(
            headless=True,
            slow_mo=self.slow_mo,
            args=["--no-sandbox", "--disable-setuid-sandbox", "--disable-dev-shm-usage"],
            timeout=30000,
        )
    else:
        # Use system Edge - always available on Windows 11, trusted by Defender
        self.browser = await self.playwright.chromium.launch(
            headless=False,
            slow_mo=self.slow_mo,
            channel="msedge",
            timeout=30000,
        )
```

4. Reports Page Performance Fix (database.py + main.py)

Problem

The View Reports page took very long to load or showed nothing at all.

Root Causes

- SQLite was in default journal mode — readers blocked during any write (e.g. while saving a test result)
- No index on `created_at` — `ORDER BY created_at DESC` performed a full table scan on every load
- No index on `test_run_id` in `test_steps` — loading step details was slow
- All DB calls were synchronous and ran directly in the async event loop, blocking it
- The sync Groq AI call blocked the event loop entirely during tests — HTTP requests for reports queued behind it

Fixes

File	Fix	Effect
<code>database.py</code>	WAL journal mode + <code>synchronous=NORMAL</code>	Reads no longer block during writes
<code>database.py</code>	<code>check_same_thread=False</code>	Safe concurrent access from thread pool
<code>database.py</code>	Index on <code>test_runs.created_at DESC</code>	List query uses index instead of full scan
<code>database.py</code>	Index on <code>test_steps.test_run_id</code>	Step detail lookup is now O(1)
<code>main.py</code>	<code>asyncio.to_thread()</code> for all DB calls	Event loop free to serve HTTP requests during DB I/O
<code>agent.py</code>	<code>asyncio.to_thread()</code> for Claude API call	Event loop free during AI inference — reports page responds instantly even mid-test

5. Event Source (SSE) Reliability Fix (main.py)

Problem

Tests running in headed mode (streaming mode) sometimes showed "Request timed out after 6 minutes" even though the test was still running.

Fix

Added a heartbeat to the SSE stream — every 25 seconds of inactivity, a comment (`: heartbeat`) is sent to keep the browser connection alive.

```
deadline = asyncio.get_event_loop().time() + 360
while True:
    remaining = deadline - asyncio.get_event_loop().time()
    if remaining <= 0:
        yield f"data: {json.dumps({'type':'error','message':'Test timed out after 6 minutes'})}\n\n"
        break
    try:
        event = await asyncio.wait_for(queue.get(), timeout=min(25, remaining))
        if event is None:
            break
        yield f"data: {json.dumps(event)}\n\n"
    except asyncio.TimeoutError:
        yield ": heartbeat\n\n" # keeps connection alive
```

6. Files Changed — Summary

File	Changes
backend/agent.py	Groq → Anthropic SDK; <code>asyncio.to_thread()</code> for API call; model set to <code>claude-haiku-4-5-20251001</code>
backend/browser.py	CSS selector detection; multi-strategy click/type; jQuery normalizer; headed mode uses <code>msedge</code> ; launch timeouts
backend/database.py	WAL mode; indexes on <code>created_at</code> and <code>test_run_id</code> ; <code>check_same_thread=False</code>
backend/main.py	<code>asyncio.to_thread()</code> for all DB calls; SSE heartbeat; API key check updated
backend/.env	Added <code>ANTHROPIC_API_KEY</code>

7. Outstanding Item

 **Anthropic account credits needed:** The API key is correctly configured and the integration is verified working (API calls reach Anthropic servers). The account currently has insufficient credits. Top up at `console.anthropic.com/settings/billing` to enable tests to run end-to-end.